ABSTACT

"Order Now" is a user-friendly food delivery application designed to connect local restaurants with customers in towns like Gunupur. The app simplifies food ordering by allowing users to browse menus, place orders, and track deliveries in real time. It empowers small food vendors by providing a digital platform to reach more customers and manage orders efficiently. With features like secure payments, real-time notifications, and delivery tracking, it enhances customer convenience and business growth. Built using modern web technologies, the app addresses challenges like limited digital presence and logistical issues, making it a reliable and scalable solution for local food delivery.

CONTENT

1. INTRODUCTION ----------1
    1.1 Purpose
    1.2 Scope
    1.3 Project Features

2. SYSTEM ANALYSIS----------3
    2.1 User Requirements
    2.2 FUNCTIONAL requirements
    2.3 Requirement Matrix

3. SYSTEM DESIGN AND SPECIFICATION---------- 6
    3.1 High Level Design
    3.2 URL Diagram
    3.3 UX Design

4. CODING----------9

5. TESTING ---------- 39
    5.1 Testing Strategy
    5.2 Test Selection
    5.3 Bug Tracking
    5.4 Reasoning
    5.5 Flowchart
    5.6 ER Diagram
    5.7 Methods
    5.8 Test Cases and Test Results

6. LIMITATION----------48

7. CONCLUSION----------51

8. REFERENCE----------53

LIST OF THE TABLE

1. Table 2.1 ----------5

LIST OF THE FIGURE

Figure 3.1 ---------- 07

Figure 5.1 ---------- 42

Figure 5.2 ---------- 43

Figure 5.3 ---------- 44

Figure 5.4 ---------- 44

Figure 5.5 ---------- 45

Figure 5.6 ---------- 45

Figure 5.7 ---------- 47

Chapter-1

INTRODUCTION

1.1 Purpose

The purpose of "OrderNow" is to create a seamless, efficient, and userfriendly online food delivery system that connects customers with restaurants. As the demand for convenience and quick access to a variety of dining options grows, OrderNow aims to bridge this gap by providing a platform where users can browse local restaurants, view menus, place orders, and track deliveries in real time. Built on the MERN stack (MongoDB, Express.js, React.js, and Node.js), OrderNow focuses on delivering a highly responsive and interactive interface that enhances the overall user experience while ensuring reliable backend performance and scalability. It enables users to explore restaurant options, customize their orders, and receive live updates on delivery status, ensuring a smooth and enjoyable experience. For restaurants, the platform opens up a digital channel that expands their reach and allows them to serve a larger audience while efficiently managing orders and optimizing operations.

1.2 Scope

OrderNow is designed to serve two primary user groups: customers and restaurant partners. Customers can explore multiple cuisine options, customize orders, and track their deliveries. Additionally, the system supports secure payment processing and personalized recommendations based on order history. For restaurant partners, OrderNow offers a comprehensive dashboard to manage menu items, view incoming orders, and access analytics to track performance. This platform has potential for scaling, including the addition of delivery partners, loyalty programs, and advanced analytics for enhanced personalization. Furthermore, it can expand to multiple cities, adapting to the

needs of various geographical locations and restaurant types. The platform is designed to be scalable, making it adaptable for

1.3 Project Features

OrderNow offers a range of features designed to deliver a comprehensive and seamless food delivery experience. It includes secure user authentication with options for social media login, allowing users to sign up and log in effortlessly. The platform provides an organized and visually engaging interface for browsing restaurants, exploring various cuisines, and viewing detailed menus. Users can customize their orders with flexibility and proceed through a smooth, secure checkout process. Real-time tracking keeps users informed about their order status and delivery location. Integrated payment gateways ensure safe and convenient online transactions. Additionally, a restaurant management dashboard allows partner restaurants to efficiently handle menu updates, manage incoming orders, and access performance analytics. Overall, OrderNow aims to be a reliable and scalable solution that connects customers with their favorite local restaurants in a quick and efficient manner.

Chapter-2

SYSTEM ANALYSIS

2.1 Users Requirements

The OrderNow online food delivery system is designed to serve two primary user groups: customers and restaurant partners, ensuring a smooth, secure, and user-focused experience. Customers seek an easy-to-use interface that allows quick navigation, access to diverse restaurant options, and straightforward ordering. They value variety in cuisine, meal types, and customization, along with a personalized experience based on their preferences and past orders. Real-time order tracking, secure multi-option payment gateways, and responsive customer support are essential for enhancing user satisfaction. On the other hand, restaurant partners require a robust platform to manage and fulfill orders efficiently.

2.2 Functional Requirements

Functional requirements are specific actions or tasks that the OrderNow system must be able to perform to meet user expectations and fulfill the purpose of the platform.

1. User Authentication and Authorization

- Sign-Up and Login: The system must support secure user registration and login, with options for social media authentication.
- Role-Based Access: Different levels of access for customers and restaurant partners to ensure data privacy and security.
- Password Recovery: Options for users to recover or reset their passwords securely.

2. Restaurant Browsing and Menu Management

- Search and Filter: Customers should be able to search for restaurants by name, cuisine, location, or popularity, and filter options based on ratings and availability. - Menu Display: Each restaurant's menu should be displayed with categories, item details, pricing, and options for customization.

- Menu Updates: Restaurant partners must be able to update menu items, availability, and pricing from their dashboard.

3. Order Placement and Customization

- Order Customization: Customers should be able to select item variants (e.g., size, spice level) and add-ons to customize their orders.
- Order Review and Confirmation: Before checkout, customers can review their order details, delivery address, and contact information.

4. Payment Processing

- Secure Payment Gateway: Integration with reliable payment providers (e.g., credit cards, debit cards, e-wallets) to ensure safe transactions.
- Order Confirmation: Automatic confirmation sent to customers and restaurant partners once the payment is processed.

5. Real-Time Order Tracking

- Order Status Updates: The system should notify customers of order status changes in real time, from preparation to out-for-delivery.
- Location Tracking: Customers can view live updates on the delivery location once the order is on the way.

6. Restaurant Dashboard and Analytics

- Order Management: Restaurant partners should have access to a dashboard where they can view and update order status.
- Performance Analytics: Analytical reports on order volume, peak times, and popular items to help restaurants improve service.

7. Customer Support

- Help Desk: A support section where customers can raise issues or queries related to orders, cancellations, and refunds.
- Feedback System: Options for customers to leave feedback or rate their experience with restaurants and delivery.

2.3 Requirement Matrix

A **Requirement Matrix** provides a clear summary of all requirements, mapping each to specific features or modules within the **OrderNow** system to ensure they are accounted for in the development process. Each requirement is categorized as a functional or non- functional requirement and assigned a priority level (high, medium, or low) based on its significance.

Requirement ID	Requirement	Description	Type	Priority
RQ-01	User Registration	Allow users to create accounts and login	Functional	High
RQ-02	Role-Based Access	Ensure distinct access levels for customers and restaurant staff	Functional	High
RQ-03	Menu Display	Display restaurant menus with details and pricing	Functional	High
RQ-04	Order Customization	Allow customers to customize orders	Functional	High
RQ-05	Payment Gateway Integration	Secure transaction options with various payment methods	Functional	High
RQ-06	Real-Time Tracking	Track order status and delivery location in real-time	Functional	High
RQ-07	Restaurant Dashboard	Interface for restaurant order and menu management	Functional	High
RQ-08	Performance Analytics	Analytical tools for restaurant partners	Functional	Medium
RQ-09	Customer Support	Help desk and feedback options for customer queries	Functional	Medium
RQ-10	Data Security	Secure storage and handling of sensitive data	Non-Functional	High
RQ-11	Scalability	Ensure the platform can handle high traffic and expandability	Non-Functional	Medium

Table 2.1

The Requirement Matrix ensures all aspects of system functionality and user expectations are addressed systematically, serving as a guiding document throughout the development process. Each requirement is tracked, prioritized, and implemented, aligning the project

Chapter-3

SYSTEM DESIGN AND SPECIFICATION

3.1 High-Level Design

The High-Level Design (HLD) of the OrderNow online food delivery system provides an overview of the architecture, key components, and their interactions within the system. OrderNow uses the MERN stack (MongoDB, Express.js, React.js, Node.js) as its primary technology stack. This design is organized into three main layers: the Frontend Layer, the Backend Layer, and the Database Layer.

Frontend Layer

The frontend is developed using React.js, creating a responsive and user-friendly interface for both customers and restaurant partners. This layer includes components for the home page, restaurant menus, order customization, cart, and real-time tracking of orders. It communicates with the backend layer through HTTP requests using RESTful APIs.

Backend Layer

The backend layer, built with Express.js and Node.js, manages business logic, handles requests, and processes transactions. It serves as the intermediary between the frontend and the database, handling operations like user authentication, order management, payment processing, and analytics. The backend follows a modular design, with separate controllers for user accounts, orders, restaurants, and payment processing.

Database Layer

The MongoDB database stores all user, restaurant, menu, order, and transaction data. MongoDB's flexible schema is particularly well-suited to accommodate the variety

of data types needed in OrderNow, such as user preferences, order history, and restaurant details. Additionally, data is stored securely, with authentication and encryption

3.2 URL Diagram

3.3 UX Design

The UX Design of OrderNow focuses on creating a simple, engaging, and intuitive user experience. The design emphasizes usability, with easy navigation and a consistent visual theme that guides users through the ordering process effortlessly. Here's a breakdown of key UX considerations:

User-Centric Interface

The UI is designed with a minimalist approach, ensuring a clutter-free environment. Buttons, icons, and menus are clearly labeled, and a logical layout allows users to find what they need with minimal effort. Essential actions, such as adding items to the cart or proceeding to checkout, are highlighted to guide users naturally through the order flow.

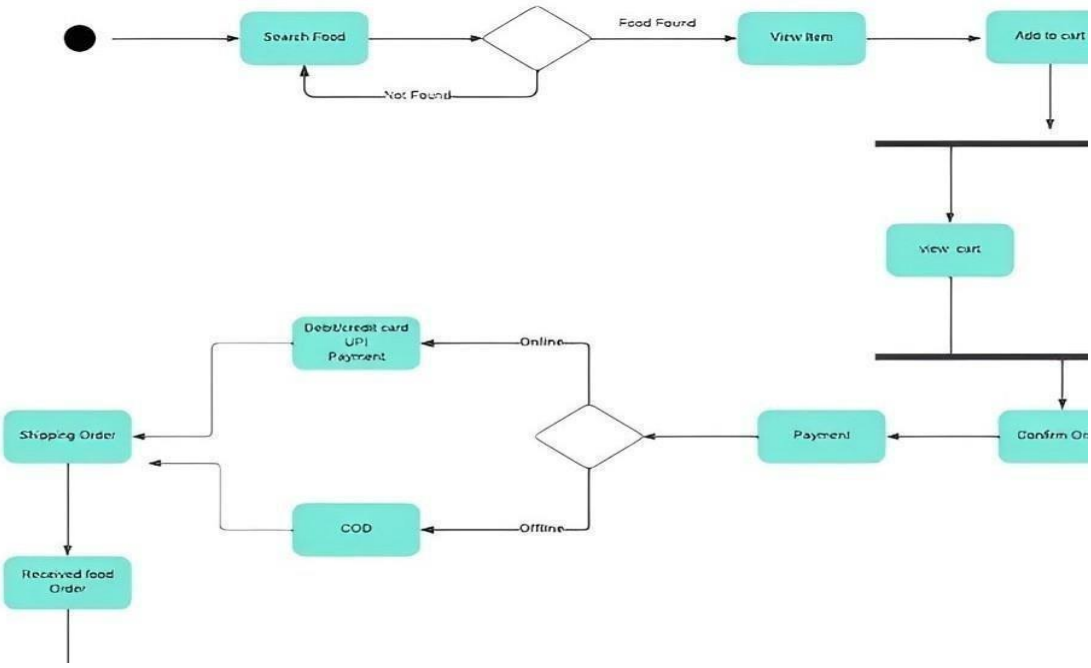


Figure 3.1

Responsive and Adaptive Design

OrderNow is designed to function smoothly across devices, including desktops, tablets, and mobile phones. Responsive layouts adjust to screen size, ensuring the platform provides the same quality of experience regardless of device. Adaptive design features, like mobile-friendly icons and gestures, make interactions smoother on smaller screens.

Visual Hierarchy

The platform uses a well-defined visual hierarchy to prioritize content. For example, restaurant listings are prominently displayed with key information like ratings, cuisine type, and distance. The menu and order customization screens are organized for easy selection and customization of food items.

User Feedback and Interaction

The platform provides immediate feedback for all user actions, such as a success message when an item is added to the cart, or a confirmation page after an order is placed. Real- time order tracking is interactive, giving users a sense of control over their delivery experience.

Streamlined Checkout Process

The checkout page is designed to be straightforward, with a single-page layout for reviewing items, adding delivery details, and making payments. Secure payment options are integrated for a seamless transaction experience, reducing abandonment rates.

Restaurant Partner Dashboard

The dashboard for restaurant partners has a clean layout with quick access to menu management, order updates, and analytics. This interface is intuitive and requires minimal training, enabling restaurant staff to handle operations smoothly.

In summary, the UX design of OrderNow is crafted to make online food ordering a pleasant and efficient experience, balancing aesthetics with functionality to meet user needs.

Chapter-4

CODING

4.1 FrontEnd

Index.html:

```
<!doctype
html>
<html
lang="en">

<head>
 <meta charset="UTF-8" />
 <link rel="icon" type="image/png" href="/dp.png" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/6.6.0/css/all.min.css"
  integrity="sha512-
Kc323vGBEqzTmouAECnVceyQqyqdsSiqLQISBL29aUW4U/M7pSPA/gEUZQqv1cw
x4OnY  xTxve5UMg5GT6L4JJg=="
  crossorigin="anonymous" referrerpolicy="no-referrer" />
 <link rel="preconnect" href="https://fonts.googleapis.com">
 <link  rel="preconnect"  href="https://fonts.gstatic.com"  crossorigin>
 <link

href="https://fonts.googleapis.com/css2?family=Poppins:ital,wght@0,100;0,200;0,300;0
,
400;0,500;0,600;0,700;0,800;0,900;1,100;1,200;1,300;1,400;1,500;1,600;1,700;1,800;1,
9
00&display=swap"
  rel="stylesheet">
 <title>ORDER NOW</title>
</head>

<body>
 <div id="root"></div>
 <script  type="module"  src="/src/main.jsx"></script>
</body>

</html>
```

Navbar.jsx import React, { useContext, useState } from "react"; import {useNavigate} from "reactrouter-dom"; import "./Navbar.css";

```
    import { assets } from
    "../../assets/assets.js"; import { Link }
    from "react-router-dom";
  import { StoreContext } from "../../context/StoreContext.jsx";

    const Navbar = ({ setShowLogin }) =>
     { const [menu, setMenu] =
     useState("home");
     const { getTotalCartAmount, token, setToken } =

     useContext(StoreContext); const navigate = useNavigate()



     const logout = () => {
       localStorage.removeItem("token
       ") setToken(""); navigate("/")
}
     return (
      <div className="navbar">
       <Link to="/">
        <img src={assets.logo} alt="" className="logo"
        />
       </Link>
       <ul className="navbar-menu">
        <Lin
         k
         to=
         "/" onClick={() =>
         setMenu("home")}
         className={menu === "home"
         ?
         "active" : ""}
        >
         Home
        </Link>



         Menu
        </a>
        <a
         href="#"
```

```
          onClick={() => setMenu("app")} className={menu
          === "app" ? "active"
          : ""} >
          App
        </a>
        <a
          href="#footer"
        <a  href="#explore-menu"  onClick={()  =>
        setMenu("menu")}  className={menu  ===
        "menu" ? "active" : ""} >
          onClick={()      =>      setMenu("contact-us")}
          className={menu === "contact-us" ? "active" :
          ""}
        >
          Contact us
        </a>
      </ul> const logout
    = () => {
      localStorage.removeItem("token
      ") setToken(""); navigate("/")
}
    return (
     <div className="navbar">
      <Link to="/">
       <img src={assets.logo} alt="" className="logo"
       />
      </Link>
      <ul className="navbar-menu">
       <Lin
        k
        to=
        "/" onClick={() =>
        setMenu("home")}
        className={menu === "home"
        ?
        "active" : ""}
       >
        Home
       </Link>
       <a  href="#explore-menu"  onClick={()  =>
       setMenu("menu")}  className={menu  ===
       "menu" ? "active" : ""} >
        Menu
       </a>
       <a
        href="#"
```

```
          onClick={() => setMenu("app")} className={menu
          === "app" ? "active"
          : ""} >
          App
         </a>
         <a
          href="#footer"
          onClick={()       =>       setMenu("contact-us")}
          className={menu === "contact-us" ? "active" :
          ""}
         >
          Contact us
         </a>
        </ul> const logout
      = () => {
        localStorage.removeItem("token
        ") setToken(""); navigate("/")
}
      return (
       <div className="navbar">
        <Link to="/">
         <img src={assets.logo} alt="" className="logo"
         />
        </Link>
        <ul className="navbar-menu">
         <Lin
          k
          to=
          "/" onClick={() =>
          setMenu("home")}
          className={menu === "home"
          ?
          "active" : ""}
         >
          Home
         </Link>
         <a  href="#explore-menu"  onClick={()  =>
         setMenu("menu")}  className={menu  ===
         "menu" ? "active" : ""} >
          onClick={()       =>       setMenu("contact-us")}
          className={menu === "contact-us" ? "active" :
          ""}
          Menu
         </a>
         <a
          href="#"
```

```
          onClick={() => setMenu("app")} className={menu
          === "app" ? "active"
          : ""} >
          App
        </a>
        <a
          href="#footer"
        >
          Contact us
        </a>
      </ul> const logout
= () => {
      localStorage.removeItem("token
      ") setToken(""); navigate("/")
}
    return (
      <div className="navbar">
        <Link to="/">
          <img src={assets.logo} alt="" className="logo"
          />
        </Link>
        <ul className="navbar-menu">
          <Lin
            k
            to=
            "/" onClick={() =>
            setMenu("home")}
            className={menu === "home"
            ?
            "active" : ""}
          >
            Home
          </Link>
          <a  href="#explore-menu"  onClick={()  =>
          setMenu("menu")}  className={menu  ===
          "menu" ? "active" : ""} >
            onClick={()      =>      setMenu("contact-us")}
            className={menu === "contact-us" ? "active" :
            ""}
          >
            Contact us
          </a>
            Menu
          </a>
          <a
            href="#"
```

```
          onClick={() => setMenu("app")} className={menu
          === "app" ? "active"
          : ""} >
          App
         </a>
         <a
          href="#footer"
        </ul> const logout
      = () => {
        localStorage.removeItem("token
        ") setToken(""); navigate("/")
}
      return (
       <div className="navbar">
        <Link to="/">
         <img src={assets.logo} alt="" className="logo"
         />
        </Link>
        <ul className="navbar-menu">
         <Lin
          k
          to=
          "/" onClick={() =>
          setMenu("home")}
          className={menu === "home"
          ?
          "active" : ""}
         >
          Home
         </Link>
         <a  href="#explore-menu"  onClick={()  =>
         setMenu("menu")}  className={menu  ===
         "menu" ? "active" : ""} >
          onClick={()      =>      setMenu("contact-us")}
          className={menu === "contact-us" ? "active" :
          ""}
         >
          Contact us
         </a>
        </ul> const logout
      = () => {

          Menu
         </a>
         <a
          href="#"
```

```
            onClick={() => setMenu("app")} className={menu
            === "app" ? "active"
            : ""} >
            App
           </a>
           <a
            href="#footer"
          localStorage.removeItem("token
          ") setToken(""); navigate("/")
}
      return (
       <div className="navbar">
        <Link to="/">
         <img src={assets.logo} alt="" className="logo"
         />
        </Link>
        <ul className="navbar-menu">
         <Lin
          k
          to=
          "/" onClick={() =>
          setMenu("home")}
          className={menu === "home"
          ?
          "active" : ""}
         >
          Home
         </Link>
         <a  href="#explore-menu"  onClick={()  =>
         setMenu("menu")}  className={menu  ===
         "menu" ? "active" : ""} >
          onClick={()      =>      setMenu("contact-us")}
          className={menu === "contact-us" ? "active" :
          ""}
         >
          Contact us
         </a>
        </ul> const logout
      = () => {
        localStorage.removeItem("token
        ") setToken(""); navigate("/")
}
           Menu
          </a>
          <a
           href="#"
```

```
      onClick={() => setMenu("app")} className={menu
      === "app" ? "active"
      : ""} >
      App
     </a>
     <a
      href="#footer"
return (
 <div className="navbar">
  <Link to="/">
   <img src={assets.logo} alt="" className="logo"
   />
  </Link>
  <ul className="navbar-menu">
   <Lin
    k
    to=
    "/" onClick={() =>
    setMenu("home")}
    className={menu === "home"
    ?
    "active" : ""}
   >
    Home
   </Link>
   <a  href="#explore-menu"  onClick={()  =>
   setMenu("menu")}  className={menu  ===
   "menu" ? "active" : ""} >
```

```
    Menu
   </a>
   <a
    href="#"
```

```
  onClick={() => setMenu("app")} className={menu
  === "app" ? "active"
  : ""} >
  App
 </a>
 <a
  href="#footer"
  onClick={()          =>          setMenu("contact-us")}
  className={menu === "contact-us" ? "active" : ""}
 >
  Contact us
 </a>
</ul>
```

```
    const logout = () => {
      localStorage.removeItem("token
      ") setToken(""); navigate("/")
}
    return (
     <div className="navbar">
      <Link to="/">
       <img src={assets.logo} alt="" className="logo" />
      </Link>
      <ul className="navbar-menu">
       <Lin
        k
        to=
        "/" onClick={() =>
        setMenu("home")}
        className={menu === "home"
        ?
        "active" : ""}
       >
        Home
       </Link>
       <a  href="#explore-menu"  onClick={()  =>
        setMenu("menu")}  className={menu  ===
        "menu" ? "active" : ""}
       >
        Menu
       </a>
       <a href="#" onClick={() =>
        setMenu("app")}
        className={menu === "app" ? "active"
        :  ""}
       >
        App
       </a>
       <a
        href="#footer"
        onClick={()          =>          setMenu("contact-us")}
        className={menu === "contact-us" ? "active" : ""}
       >
        Contact us
       </a>
      </ul>
```

```
  <div className="navbar-right">
    <i className="fa-solid fa-magnifying-glass"></i>
    <div  className="navbar-search-icon">
      <Link to="/cart">
        <i className="fa-solid fa-cart-shopping"></i>
      </Link>
<div className={getTotalCartAmount() === 0 ? "" : "dot"}></div>
    </div>
    {!token ? (
      <button onClick={() => setShowLogin(true)}>Sign in</button>
```

```
      ) : (
        <div className="navbar-profile">
          <img src={assets.profile_icon} alt="" />
          <ul className="nav-profile-dropdown">
            <li onClick={() => navigate('/myorders')}>
              <i className="fa-solid fa-bag-shopping"></i>
              <p>Orders</p>
            </li>
            <hr />
            <li onClick={logout}>
            <i className="fa-solid fa-right-from-bracket"></i>
              <p>Logout</p>
            </li>
          </ul>
        </div>
      )}
    </div>
  </div>
 );
};

export default Navbar;
```

Header.jsx

```
import React
from "react";
import
"./Header.css";

const Header = ()
  => { return (
    <>
      <div className="Header">
        <div className="header-contents">
          <h2>Order your happiness</h2>
          <p>
            Lorem ipsum dolor sit amet consectetur adipisicing elit.
            Quisquam quia facere illo suscipit enim rem?
          </p>
          <button>View Menu</button>
        </div>
      </div>
```

```
    </>
  );
};

export default Header;
```

ExploreMenu.jsx

```
import React from
'react' import './ExploreMenu.css' import
{ menu_list } from '../../assets/assets'
```

```
const ExploreMenu = ({category,setCategory})
  => { return (
    <div className='explore-menu' id='explore-menu'>
       <h1>Explore our menu</h1>
       <p classNam='explore-menu-text'>Lorem ipsum dolor sit amet consectetur
adipisicing elit.</p>
       <div className=''explore-menu-list">
          {menu_list.map((item,inde
             x)=> { return (
                <div          onClick={()          =>
                   setCategory(prev =>
                      prev===item.menu_name
                      ? "All"
                      : item.menu_name
                   )}

                   key={index}
                   className='explore-menu-list-item'

                   >
                   <img className={category === item.menu_name ? "active" : ""}
src={item.menu_image} alt="" />
                   <p>{item.menu_name}</p>
                </div>
      )
    })}
</div>
<hr />
     </div>
  )
}

export default ExploreMenu
```

FoodDisplay.jsx:

```
import React, { useContext } from
"react"; import "./FoodDisplay.css";
import { StoreContext } from
"../../context/StoreContext"; import FoodItem from
"../FoodItem/FoodItem";
```

```
const FoodDisplay = ({ category }) => {
  const { food_list } =
  useContext(StoreContext); return (
    <div className="food-display" id="food-display">
      <h2>Top dishes near you</h2>
      <div className="food-display-list">
        {food_list.map((item, index) => { if
          (category === "All" || category ===
          item.category) { return (
            <FoodItem
              key={inde
              x}
              id={item._i
              d}
```

```
            name={item.name}
            description={item.descri
            ption} price={item.price}
            img={item.image}
          />
        );
      }
    })}
   </div>
  </div>
 );
};

export default

FoodDisplay;
```

4.2BackEnd

FoodModel.js

```
import mongoose from "mongoose";

const foodSchema = new
  mongoose.Schema({ name: {
    type:
    String,
    require
    d: true
  },
  descriptio
    n: {
    type:
    String,
    require
    d: true
  },
  price: {
    type: Number,
    required: true
  },
```

```
  image: {
    type:
    String,
    require
    d: true
  },
  category:
    {  type:
    String,
    require
    d: true
  }
})

const foodModel = mongoose.models.food || mongoose.model("food",

foodSchema) export default foodModel;
```

UserModel.js

```
import mongoose from 'mongoose'

const userSchema = new
  mongoose.Schema({
  name:{ type: String,
  require
    d: true
  },
  email: {
    type:    String,
    required: true,
    unique: true
  },
  password
    : { type:
    String,
    require
    d: true
  },
  cartDat
    a: {
    type:
    Object
    ,
    defaul
    t: {}
  }
},{minimize:false})

const userModel = mongoose.models.user || mongoose.model("user",

userSchema); export default userModel;
```

OrderModel.js

```
import mongoose from 'mongoose'

const orderSchema = new
```

```
  mongoose.Schema({ userId:{
    type:                String,
    required:true,
  },
  items:{
    type:Ar
    ray,
    require
    d:true
  },
  amount:{
    type:            Number,
    required: true
  },
  address:
    {
    type:
    Object
    ,
    required: true
  },

const removeFromCart = async
  (req,res) => { try { let
    userData = await
    userModel.findById(req.body.userId) let cartData
    = await userData.cartData; if
    (cartData[req.body.itemId] > 0) {
      cartData[req.body.itemId] -= 1;
    }

const removeFromCart = async
  (req,res) => { try { let
    userData = await
    userModel.findById(req.body.userId) let cartData
    = await userData.cartData; if
    (cartData[req.body.itemId] > 0) {
      cartData[req.body.itemId] -= 1;
    }

const removeFromCart = async
  (req,res) => { try { let
    userData = await
```

```
        userModel.findById(req.body.userId) let cartData
        = await userData.cartData; if
        (cartData[req.body.itemId] > 0) {
          cartData[req.body.itemId] -= 1;
        }
type:       String,       default:"Food
processing"
      },
      date:{
        type:Date,
        default: Date.now()
      },
      payment:{
        type:         Boolean,
        default: false
      }
    })

    const    orderModel    =    mongoose.models.order    ||    mongoose.model("order",

    orderSchema) export default orderModel;
```

CartControl.js

```
    import userModel from '../models/userModel.js'

    //add items to user cart

    const addToCart = async
      (req,res) => { try { let
        userData = await
        userModel.findById(req.body.userId); let cartData =
        await userData.cartData;
        if(!cartData[req.body.itemId]){
          cartData[req.body.itemId] = 1;
        }
        else{
          cartData[req.body.itemId] += 1;
        }

        await userModel.findByIdAndUpdate(req.body.userId,{cartData});
        res.json({success: true, message: "Added to cart"})
```

```
  } catch (error) {
    console.log(err
    or)
    res.json({success: false, message: "Error"})
  }
}

//remove items from user cart


const removeFromCart = async
  (req,res) => { try { let
    userData = await
    userModel.findById(req.body.userId) let cartData
    = await userData.cartData; if
    (cartData[req.body.itemId] > 0) {
      cartData[req.body.itemId] -= 1;
    }
```

```
        await userModel.findByIdAndUpdate(req.body.userId, {cartData})
        res.json({success: true, message: "Removed from cart"})

      } catch (error) {
        console.log(err
        or);
        res.json({success: false, message: "Error"})
      }
    }

    //fetch user cart data

    const getCart = async (req,res)
      => { try { let
        userData = await
        userModel.findById(req.body.userId); let cartData
        = await userData.cartData; res.json({success: true,
        cartData})
      } catch (error) {
        console.log(err
        or);
        res.json({success: false, message: "Error"})
      }
    }

    export { addToCart, removeFromCart, getCart }

@REM ----------------------------------------------------------------------------
@REM Licensed to the Apache Software Foundation (ASF) under one
@REM or more contributor license agreements.  See the NOTICE file
@REM distributed with this work for additional information
@REM regarding copyright ownership.  The ASF licenses this file
@REM to you under the Apache License, Version 2.0 (the
@REM "License"); you may not use this file except in compliance
@REM with the License.  You may obtain a copy of the License at
@REM
@REM    https://www.apache.org/licenses/LICENSE-2.0
@REM
@REM Unless required by applicable law or agreed to in writing,
@REM software distributed under the License is distributed on an
@REM "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
```

```
@REM KIND, either express or implied.  See the License for the
@REM specific language governing permissions and limitations
@REM under the License.
@REM ----------------------------------------------------------------------------

@REM ----------------------------------------------------------------------------
@REM Maven Start Up Batch script
@REM
@REM Required ENV vars:
@REM JAVA_HOME - location of a JDK home dir
@REM
@REM Optional ENV vars
@REM M2_HOME - location of maven2's installed home dir
@REM MAVEN_BATCH_ECHO - set to 'on' to enable the echoing of the batch
commands
@REM MAVEN_BATCH_PAUSE - set to 'on' to wait for a keystroke before ending
@REM MAVEN_OPTS - parameters passed to the Java VM when running Maven
@REM     e.g. to debug Maven itself, use
@REM set MAVEN_OPTS=-Xdebug -
Xrunjdwp:transport=dt_socket,server=y,suspend=y,address=8000
@REM MAVEN_SKIP_RC - flag to disable loading of mavenrc files
@REM ----------------------------------------------------------------------------

@REM Begin all REM lines with '@' in case MAVEN_BATCH_ECHO is 'on'
@echo off
@REM set title of command window
title %0
@REM enable echoing by setting MAVEN_BATCH_ECHO to 'on'
@if "%MAVEN_BATCH_ECHO%" == "on"  echo %MAVEN_BATCH_ECHO%

@REM set %HOME% to equivalent of $HOME
if "%HOME%" == "" (set "HOME=%HOMEDRIVE%%HOMEPATH%")

@REM Execute a user defined script before this one
if not "%MAVEN_SKIP_RC%" == "" goto
skipRcPre
@REM check for pre script, once with legacy .bat ending and once with .cmd ending if
exist "%USERPROFILE%\mavenrc_pre.bat" call "%USERPROFILE%\mavenrc_pre.bat"
%* if exist "%USERPROFILE%\mavenrc_pre.cmd" call
"%USERPROFILE%\mavenrc_pre.cmd" %* :skipRcPre

@setlocal
```

```
set ERROR_CODE=0

@REM To isolate internal variables from possible post scripts, we use another setlocal
@setlocal

@REM ==== START VALIDATION ====
if not "%JAVA_HOME%" == "" goto OkJHome

echo.
echo Error: JAVA_HOME not found in your environment. >&2
echo Please set the JAVA_HOME variable in your environment to match the
>&2 echo location of your Java installation. >&2 echo.
goto error

:OkJHome
if exist "%JAVA_HOME%\bin\java.exe" goto init

echo. echo Error: JAVA_HOME is set to an invalid
directory. >&2
echo JAVA_HOME = "%JAVA_HOME%" >&2
echo Please set the JAVA_HOME variable in your environment to match the
>&2 echo location of your Java installation. >&2 echo.
goto error

@REM ==== END VALIDATION ====

:init

@REM Find the project base dir, i.e. the directory that contains the folder ".mvn".
@REM Fallback to current working directory if not found.

set MAVEN_PROJECTBASEDIR=%MAVEN_BASEDIR%
IF NOT "%MAVEN_PROJECTBASEDIR%"=="" goto endDetectBaseDir

set EXEC_DIR=%CD%
set WDIR=%EXEC_DIR%
:findBaseDir
IF EXIST "%WDIR%"\.mvn goto baseDirFound
cd ..
IF "%WDIR%"=="%CD%" goto baseDirNotFound
set WDIR=%CD%
goto findBaseDir
```

```
:baseDirFound
set MAVEN_PROJECTBASEDIR=%WDIR%
cd "%EXEC_DIR%"
goto endDetectBaseDir

:baseDirNotFound
set MAVEN_PROJECTBASEDIR=%EXEC_DIR%
cd "%EXEC_DIR%"

:endDetectBaseDir

IF NOT EXIST "%MAVEN_PROJECTBASEDIR%\.mvn\jvm.config" goto endReadAdditionalConfig

@setlocal EnableExtensions EnableDelayedExpansion
for /F "usebackq delims=" %%a in ("%MAVEN_PROJECTBASEDIR%\.mvn\jvm.config") do set JVM_CONFIG_MAVEN_PROPS=!JVM_CONFIG_MAVEN_PROPS! %%a
@endlocal & set JVM_CONFIG_MAVEN_PROPS=%JVM_CONFIG_MAVEN_PROPS%

:endReadAdditionalConfig

SET MAVEN_JAVA_EXE="%JAVA_HOME%\bin\java.exe"
set WRAPPER_JAR="%MAVEN_PROJECTBASEDIR%\.mvn\wrapper\maven-wrapper.jar" set WRAPPER_LAUNCHER=org.apache.maven.wrapper.MavenWrapperMain

set DOWNLOAD_URL="https://repo.maven.apache.org/maven2/org/apache/maven/wrapper/maven-wrapper/3.1.0/maven-wrapper-3.1.0.jar"

FOR /F "usebackq tokens=1,2 delims==" %%A IN ("%MAVEN_PROJECTBASEDIR%\.mvn\wrapper\maven-wrapper.properties") DO (
    IF "%%A"=="wrapperUrl" SET DOWNLOAD_URL=%%B
)

@REM Extension to allow automatically downloading the maven-wrapper.jar from Mavencentral
@REM This allows using the maven wrapper in projects that prohibit checking in binary data.
if exist %WRAPPER_JAR% ( if "%MVNW_VERBOSE%" == "true"
(
```

```
    echo Found %WRAPPER_JAR%
  )
) else (
  if not "%MVNW_REPOURL%" == "" (
    SET
DOWNLOAD_URL="%MVNW_REPOURL%/org/apache/maven/wrapper/mavenwrap
per/3.1.0/maven-wrapper-3.1.0.jar"
  )
  if "%MVNW_VERBOSE%" == "true" (
    echo
Couldn't find %WRAPPER_JAR%, downloading it ...
    echo Downloading from: %DOWNLOAD_URL%
  )

  powershell -Command "&{"^
    "$webclient = new-object System.Net.WebClient;"^
    "if (-not ([string]::IsNullOrEmpty('%MVNW_USERNAME%') -and
[string]::IsNullOrEmpty('%MVNW_PASSWORD%'))) {"^
    "$webclient.Credentials = new-object
System.Net.NetworkCredential('%MVNW_USERNAME%',
'%MVNW_PASSWORD%');"^
    "}"^
    "[Net.ServicePointManager]::SecurityProtocol =
[Net.SecurityProtocolType]::Tls12; $webclient.DownloadFile('%DOWNLOAD_URL%',
'%WRAPPER_JAR%')"^
    "}"
  if "%MVNW_VERBOSE%" == "true" (
    echo Finished downloading %WRAPPER_JAR%
  )
)
@REM End of extension

@REM Provide a "standardized" way to retrieve the CLI args that will
@REM work with both Windows and non-Windows executions.
set MAVEN_CMD_LINE_ARGS=%*

%MAVEN_JAVA_EXE% ^
 %JVM_CONFIG_MAVEN_PROPS% ^
 %MAVEN_OPTS% ^
 %MAVEN_DEBUG_OPTS% ^
 -classpath %WRAPPER_JAR% ^
 "-Dmaven.multiModuleProjectDirectory=%MAVEN_PROJECTBASEDIR%" ^
 %WRAPPER_LAUNCHER% %MAVEN_CONFIG% %*
```

```
if ERRORLEVEL 1 goto error
goto end

:error
set ERROR_CODE=1

:end
@endlocal & set ERROR_CODE=%ERROR_CODE%

if not "%MAVEN_SKIP_RC%"=="" goto skipRcPost
@REM check for post script, once with legacy .bat ending and once with .cmd ending if
exist "%USERPROFILE%\mavenrc_post.bat" call
"%USERPROFILE%\mavenrc_post.bat" if exist
"%USERPROFILE%\mavenrc_post.cmd" call "%USERPROFILE%\mavenrc_post.cmd"
:skipRcPost

@REM pause the script if MAVEN_BATCH_PAUSE is set to 'on'
if "%MAVEN_BATCH_PAUSE%"=="on" pause

if "%MAVEN_TERMINATE_CMD%"=="on" exit %ERROR_CODE%

cmd /C exit /B %ERROR_CODE%

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-parent</artifactId>
                <version>2.7.0</version>
                <relativePath/> <!-- lookup parent from repository -->
        </parent>
        <groupId>com.example</groupId>
        <artifactId>food-order-back</artifactId>
        <version>0.0.1-SNAPSHOT</version>
        <name>food-order-back</name>
        <description>Demo project for Spring Boot</description>
        <properties>
                <java.version>17</java.version>
        </properties>
```

```
<dependencies>
	<dependency>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-data-jpa</artifactId>
	</dependency>

	<dependency>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-validation</artifactId>
	</dependency>
	<dependency>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-web</artifactId>
	</dependency>

	<dependency>
		<groupId>mysql</groupId>
		<artifactId>mysql-connector-java</artifactId>
		<scope>runtime</scope>
	</dependency>
	<dependency>
		<groupId>org.projectlombok</groupId>
		<artifactId>lombok</artifactId>
		<optional>true</optional>
	</dependency>
	<dependency>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-test</artifactId>
		<scope>test</scope>
	</dependency>
	<dependency>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-security</artifactId>
	</dependency>
	<dependency>
		<groupId>org.springframework.security</groupId>
		<artifactId>spring-security-test</artifactId>
		<scope>test</scope>
	</dependency>
	<dependency>
		<groupId>io.jsonwebtoken</groupId>
		<artifactId>jjwt</artifactId>
		<version>0.4</version>
```

```
            </dependency>
            <dependency>
          <groupId>com.googlecode.json-simple</groupId>
           <artifactId>json-simple</artifactId>
           <version>1.1.1</version>
            </dependency>
            <dependency>
            <groupId>com.google.code.gson</groupId>
             <artifactId>gson</artifactId>
             <version>2.8.9</version>
           </dependency>
        </dependencies>

        <build>
            <plugins>
                <plugin>
                    <groupId>org.springframework.boot</groupId>
                    <artifactId>spring-boot-maven-plugin</artifactId>
                    <configuration>
                        <excludes>
                            <exclude>
                                <groupId>org.projectlombok</groupId>
                                <artifactId>lombok</artifactId>
                            </exclude>
                        </excludes>
                    </configuration>
                </plugin>
            </plugins>
        </build>

</project>
```

Chapter-5

TESTING

5.1 Testing Strategy

The **Testing Strategy** for **OrderNow** involves a comprehensive approach to verify and validate that the system meets user requirements, functions as intended, and provides a seamless experience. The testing approach includes both **functional testing** and **non- functional testing** to cover all system aspects.

- **Functional Testing**: Focuses on verifying that each feature, such as user authentication, order management, payment processing, and real-time tracking, works according to specifications.
- **Non-Functional Testing**: Ensures system scalability, performance, usability, and security. This includes load testing, stress testing, and security testing to confirm the system's robustness.
- **Automated Testing**: To ensure efficiency, automated tests are used for repetitive tasks, like regression testing, while manual testing is employed for exploratory and usability testing.

This strategy ensures that OrderNow is thoroughly tested in all critical areas to achieve high quality and reliability.

5.2 Test Selection

Test Selection identifies which tests to perform based on the criticality and functionality of each feature in OrderNow. Here are the key test types chosen:

- **Unit Testing**: Testing individual components (e.g., menu display, order customization) to verify each part works independently.
- **Integration Testing**: Ensures that components work together smoothly. For example, testing the integration between the frontend (React) and backend (Node.js) APIs.

- **System Testing**: Validates the entire OrderNow platform to check if it meets the specified requirements. This includes end-to-end testing from browsing to checkout and order tracking.
- **Acceptance Testing**: Confirms the system is ready for release by verifying it meets all user requirements, based on real-world scenarios.
- **Regression Testing**: Re-testing previously successful functions after any updates to ensure new code changes haven't caused errors in existing functionality.

5.3 Bug Tracking

Bug Tracking is essential for identifying, managing, and resolving issues during the testing process. For OrderNow, a bug tracking tool such as **Jira** or **Trello** is used to track bugs throughout the development cycle. Each bug report contains:

- **Bug ID**: A unique identifier for each bug.
- **Description**: Detailed explanation of the issue, including steps to reproduce it.
- **Severity and Priority**: Classification of the bug's impact on the system.
- **Status**: Indicates if the bug is open, in progress, or resolved.
- **Assigned Developer**: Specifies who is responsible for fixing the bug.
- **Resolution**: Description of how the bug was fixed, once resolved.

This organized bug tracking system helps prioritize critical issues, streamline communication between testers and developers, and maintain a record of resolved issues for future reference.

5.4 Reasoning

The **Reasoning** behind the testing approach for OrderNow is to ensure quality, reliability, and a positive user experience. Given the nature of online food delivery systems, critical areas include user authentication, order management, payment security, and real-time order tracking. The reasoning behind specific testing strategies is as follows:

- **User-Centric Approach**: Testing focuses on user-critical features such as order customization, secure payments, and order tracking to ensure smooth functionality.
- **Data Security**: As OrderNow handles sensitive user and payment data, security testing is prioritized to protect against potential vulnerabilities.
- **Performance Under Load**: Load testing is crucial for ensuring that the system can handle a large number of simultaneous users, especially during peak times.

These reasons guide the testing strategy, aiming to deliver a high-quality, reliable product that meets both user expectations and technical standards.

5.5 Flowchart

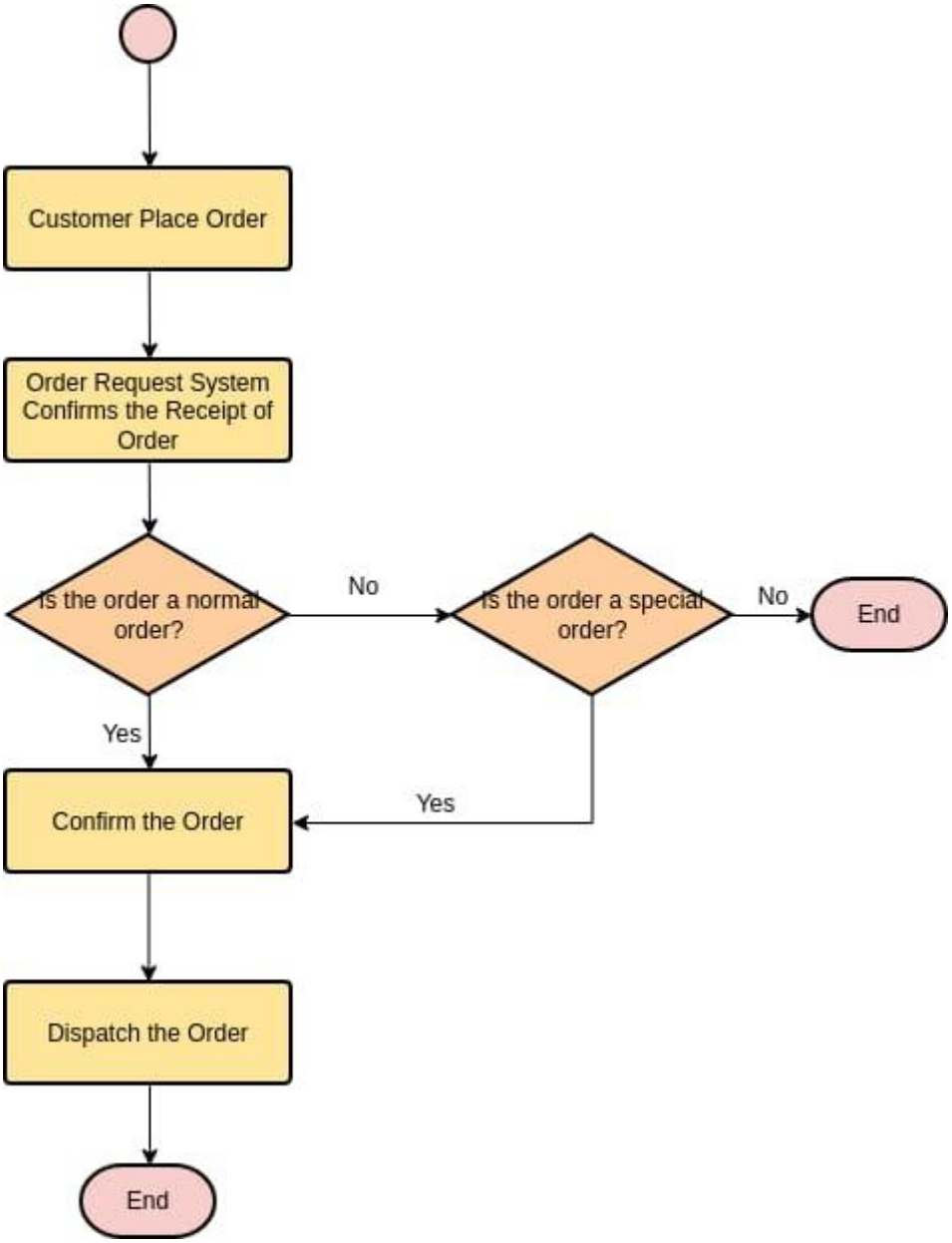


Figure 5.1

5.6 ER Diagram

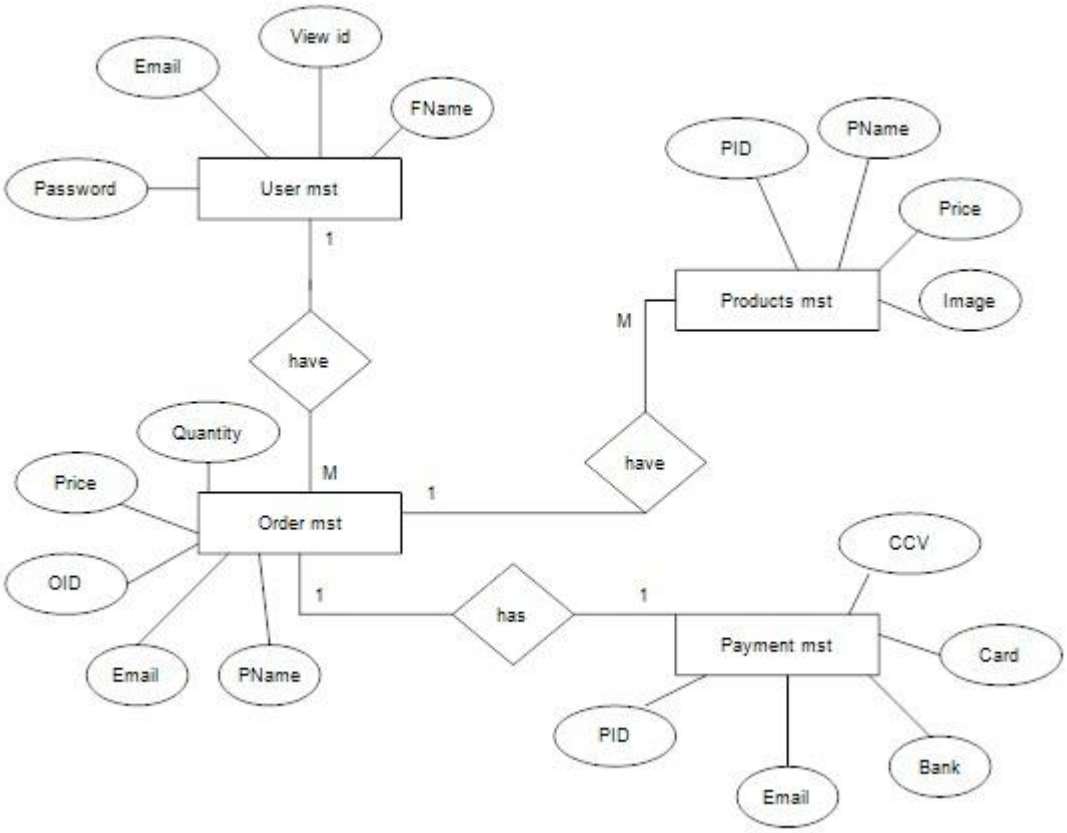


Figure 5.2

5.7 Methods

The **Testing Methods** employed for OrderNow include:

- **Manual Testing**: Used for exploratory, usability, and acceptance testing. This helps evaluate real-user interactions and gather insights for user experience improvements.

- **Automated Testing**: Automated tools like **Selenium** or **Cypress** are used for regression testing, ensuring previously tested functionalities continue to work as expected after updates.
- **Black-Box Testing**: Focuses on testing system functionality without knowledge of internal code structure, simulating a real user's interaction with the system.
- **White-Box Testing**: Applied selectively for code optimization and verification of complex functionalities, ensuring efficient and correct logic flow.

5.8 Test Cases and Test

Results Home page

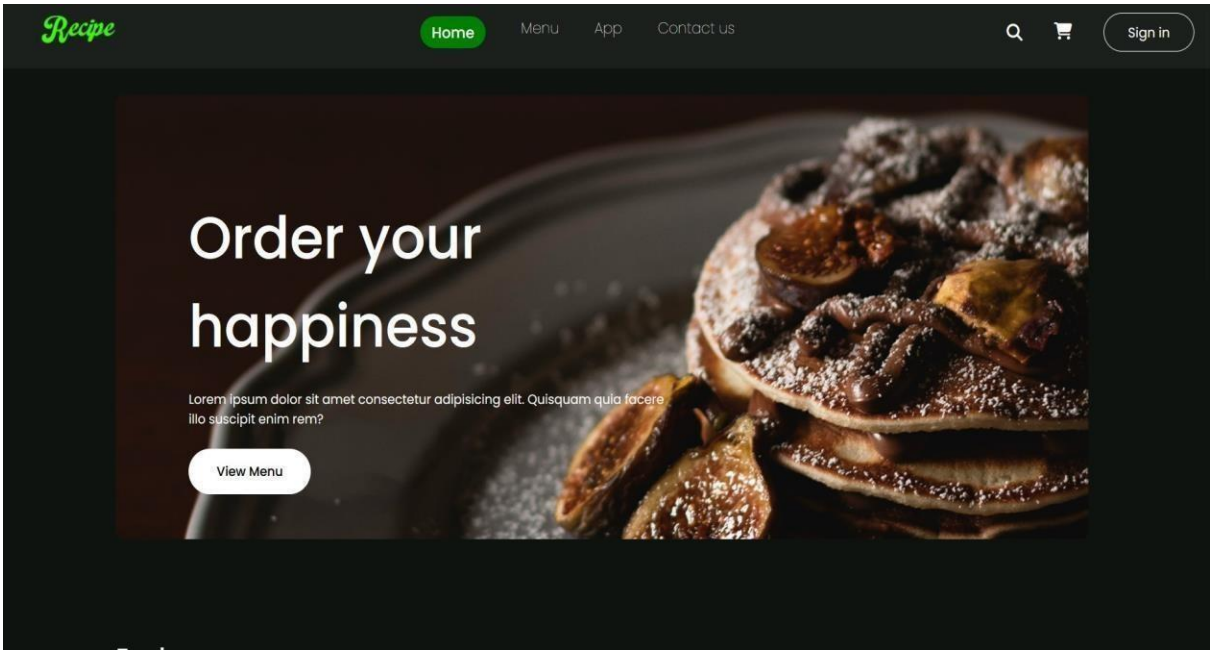


Figure 5.3

Food Items

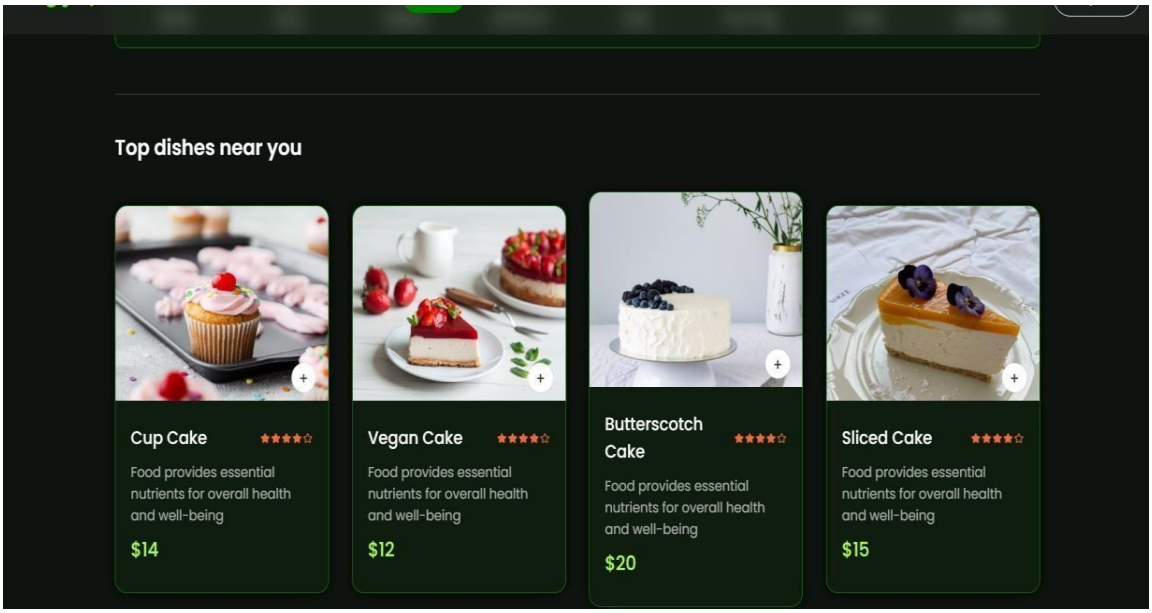


Figure 5.4



Figure 5.5

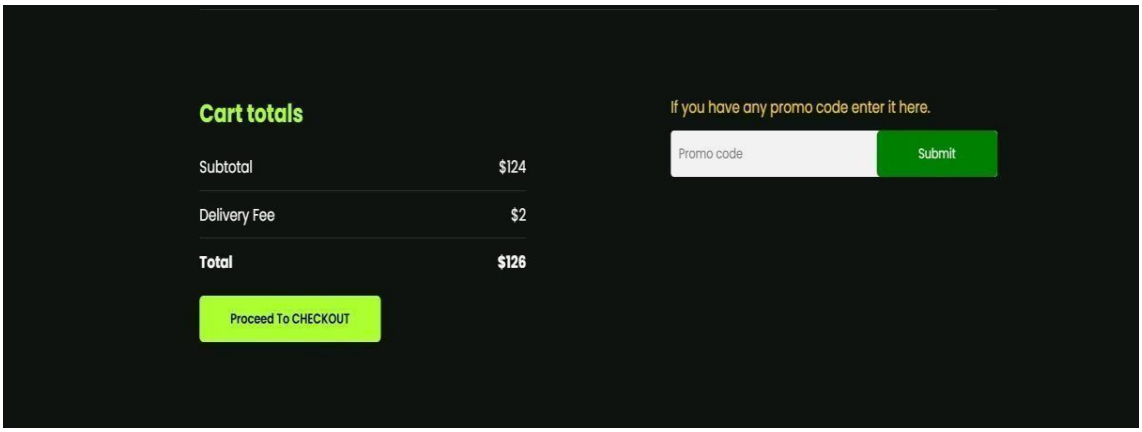


Figure 5.6

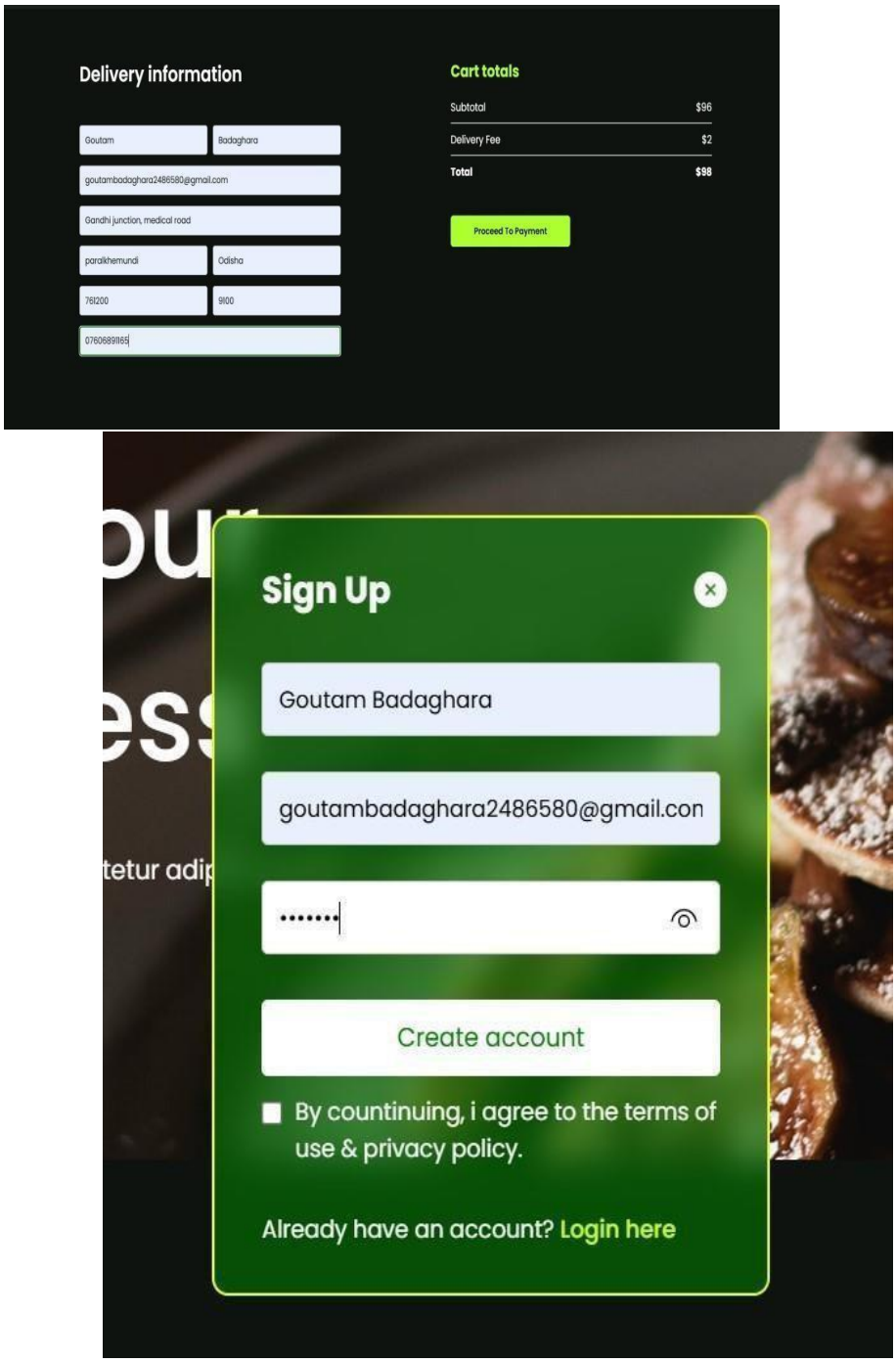

Delivery information
Goutam
Badaghara
goutambadaghara2486580@gmail.com
Gandhi junction, medical road
paralkhemundi
Odisha
761200
9100
07606891165
Cart totals
Subtotal
$96
Delivery Fee
$2
Total
$98
Proceed To Payment
Sign Up
Goutam Badaghara
goutambadaghara2486580@gmail.con
Create account
By countinuing, i agree to the terms of use & privacy policy.
Already have an account? Login here

Figure 5.7

Chapter-6

LIMITATION

While the OrderNow Online Food Delivery System developed using the MERN stack is designed to provide a smooth and efficient food delivery experience, several limitations have been identified that could affect system performance, user experience, and scalability. Addressing these limitations in future updates or system upgrades will be essential for enhancing the application's overall performance and user satisfaction.

1. Scalability Constraints

Although the MERN stack is effective for building scalable applications, scaling this system for high-traffic volumes could be challenging. The current setup may not be sufficient to handle large spikes in user demand, especially during peak times, which could lead to slower response times or temporary downtime. Upgrading to a more distributed, microservices architecture might help improve scalability but would require significant redevelopment efforts.

2. Dependency on Internet Connection

The OrderNow system is a web-based application that requires a stable internet connection to function effectively. Users with limited or inconsistent internet connectivity may face issues accessing the platform, particularly in remote areas. Offline functionality, such as viewing previous orders or saving menus for offline browsing, is not currently available, which could impact accessibility for some users.

3. Limited Payment Options

The current system integrates only a few common payment gateways, potentially restricting users who rely on less popular payment methods. Expanding payment options to include digital wallets, regional payment methods, and cryptocurrency could enhance the platform’s appeal to a broader

4. Real-Time Tracking Delays

Real-time tracking is a core feature, but limitations in tracking accuracy and update frequency may lead to delays in displaying the delivery status. For example, if a delivery is made in a poor connectivity area, updates might not be as precise. Integrating advanced tracking technologies or working closely with delivery service providers could improve the accuracy of this feature, but it would increase operational costs.

5. Dependency on Partner Restaurants

OrderNow's functionality and user satisfaction heavily depend on partner restaurants keeping their menus, availability, and inventory updated. Miscommunication or delays in updates from restaurants could result in inaccurate information on the platform, leading to user frustration. Implementing automated tools for restaurant partners to update their profiles or inventories could reduce these inconsistencies.

6. User Interface Limitations

While the interface is designed to be responsive, some complex visual elements may not display optimally on all devices, particularly older mobile models or smaller screens. Additionally, with a limited number of language options, non- English speakers may find it challenging to navigate the system. A more universally accessible design with multilingual support would be beneficial, though it would require additional resources for development and localization.

7. Privacy and Data Security Concerns

As a platform that handles user data, including payment information, privacy and security are critical. Despite implementing industry-standard security measures, there is always a risk of data breaches. OrderNow's current security protocols may require frequent updates to stay aligned with new security standards, which could increase ongoing maintenance costs.
improve engagement but would require additional development and potentially more server resources to handle these interactions effectively.

In summary, while OrderNow fulfills its primary function as an online food delivery platform, addressing these limitations will be essential to meet user expectations, improve reliability, and increase the platform's scalability. Future updates could include enhancements to the architecture, broader payment integration, and additional user-centered features to overcome these limitations and create a more robust, adaptable system.

Chapter-7

CONCLUSION AND FUTURE SCOPE

The **OrderNow Online Food Delivery System** successfully meets the needs of modern consumers by providing a streamlined platform for ordering food from a diverse range of restaurants. Built on the MERN stack, it combines efficiency, a responsive user interface, and essential features such as order customization, secure payments, and real-time tracking, ensuring a highquality user experience. The design's modularity and use of RESTful APIs support scalability and maintainability, positioning OrderNow as a reliable solution for both customers and restaurant partners. Despite these strengths, current limitations include scalability constraints, dependency on stable internet connectivity, and limited payment options, highlighting areas where the platform can evolve.

Future Scope

To enhance OrderNow's performance and adaptability, several future developments are proposed:

1. **Scalability Improvements**: Transitioning to a microservices architecture could allow for higher traffic handling, especially during peak times. This approach would enable the independent scaling of services, optimizing resource usage.
2. **Expanded Payment Options**: Integrating additional payment gateways, digital wallets, and even cryptocurrency options would broaden accessibility for users, supporting a variety of regional preferences.
3. **Advanced Real-Time Tracking**: Implementing GPS-based delivery tracking with more precise, real-time location updates would enhance

the delivery experience for users, especially in densely populated or connectivity-limited areas.

4. **Offline Functionality**: Developing offline capabilities, such as menu browsing and order history access, would make the platform more accessible to users with inconsistent internet.
5. **Multilingual Support**: Adding multiple language options would improve accessibility, accommodating non-English speakers and increasing the platform's reach across diverse demographics.

By implementing these enhancements, OrderNow can better meet evolving user needs and establish itself as a leading solution in the online food delivery market.

REFERENCES AND BIBLOGRAPHY

[1] Hamlin, M.R.A., & Mayan, J.A. (2016). Blood Donation and Life Saver - Blood Donation App. Proceedings of the International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT), Kumaracoil, India, pp. 625–628. https://doi.org/10.1109/ICCICCT.2016.798802

[2] Meiappane, A., et al. (2019). DWORLD: Blood Donation App Using Android. Proceedings of the IEEE International Conference on System, Computation, Automation and Networking (ICSCAN), Pondicherry, India, pp. 1–5. https://doi.org/10.1109/ICSCAN.2019.8878830

[3] Pohandulkar, S.S., & Khandelwal, C.S. (2018). [Title of the Paper]. [Conference/Journal Name] [Location], [Pages if available]. [DOI if available]

Resources and Websites -

https://www.javatpoint.com/
https://www.javatpoint.com/
https://www.youtube.com/